

Gestión de Configuración de Software

O de por qué un commit no es un sistema de control.

UTN · Facultad Regional Córdoba

Cátedra de Ingeniería y Calidad de Software

Docentes: Judith Meles — Laura Covaro

Elaborado a partir de la presentación «Software Configuration Management (SCM)» (58 láminas)

CONTENIDO

1. El software en contexto
2. El software cambia: problemas típicos
3. La SCM como disciplina de soporte
4. Definición y propósito: la integridad del producto
5. Conceptos clave: ítem, versión, variante, configuración
6. El repositorio
7. Líneas base
8. Ramas
9. Las cuatro actividades de la SCM
10. Identificación de ítems de configuración
11. Control de cambios
12. Auditorías de configuración
13. Registro e informes de estado
14. El plan de gestión de configuración
15. SCM en ambientes ágiles
16. Preguntas de repaso
17. Bibliografía

ANTES DE EMPEZAR: LA SIGLA DE LA UNIDAD

SCM — *Software Configuration Management*. En castellano se la conoce como **Gestión de Configuración de Software** (algunos textos, como la lámina 10, dicen «Administración de Configuración de Software»; es lo mismo). Ojo con la traducción literal «manejo de la configuración»: no se trata de configurar parámetros de un programa, sino de **controlar qué artefactos componen el producto, en qué versión y cómo cambian**. A lo largo del apunte cada sigla se aclara una sola vez, la primera vez que aparece.

01 El software en contexto

lám. 2–4

La presentación arranca ubicando al software dentro de un sistema más grande. Las **personas** se incorporan a un **proyecto**; el proyecto se adapta a un **proceso**, que a su vez se automatiza con **herramientas**; y el resultado del proyecto es un **producto**. La SCM aparece justamente en el cruce de todo eso: es un proceso, se apoya fuertemente en herramientas, la ejecutan personas y su objeto de cuidado es el producto.



Lámina 2. Personas, proceso, proyecto, herramientas y producto: el contexto donde vive el software.

Cuando pensamos en software solemos pensar en código, pero la cátedra retoma una definición más amplia que la de la Unidad 1. El software es un conjunto de:

- **Programas** (fuentes, objetos, ejecutables).
- **Procedimientos** (cómo se instala, se construye, se opera).
- **Reglas** (de negocio, de codificación, de nombrado).
- **Documentación** (requerimientos, diseño, manuales).
- **Datos** (de prueba, de configuración, de migración).

Y se lo caracteriza como **información**: estructurada con propiedades lógicas y funcionales, creada y mantenida en varias formas y representaciones, y confeccionada para ser procesada por computadora en su estado más desarrollado. Esta idea es la puerta de entrada a la unidad: si el software es información que existe en muchas formas a la vez (un requerimiento, un diseño, un fuente, un objeto), **cada una de esas formas puede cambiar por separado y desincronizarse de las demás**.

02 El software cambia: problemas típicos

Lám. 5–8, 14–15

Las láminas 5 a 7 muestran el mismo árbol en dos momentos. En enero hay un requerimiento, un diseño, seis fuentes y tres objetos, todos coherentes entre sí. En febrero cambia el requerimiento, eso obliga a tocar el diseño, algunos fuentes y algunos objetos. La pregunta de la lámina 7 es la clave: **¿esto que tengo en la mano es realmente «febrero»?** Si un fuente quedó con la versión de enero o un objeto no se recompiló, el sistema es una mezcla que nadie sabe describir.

Problemas

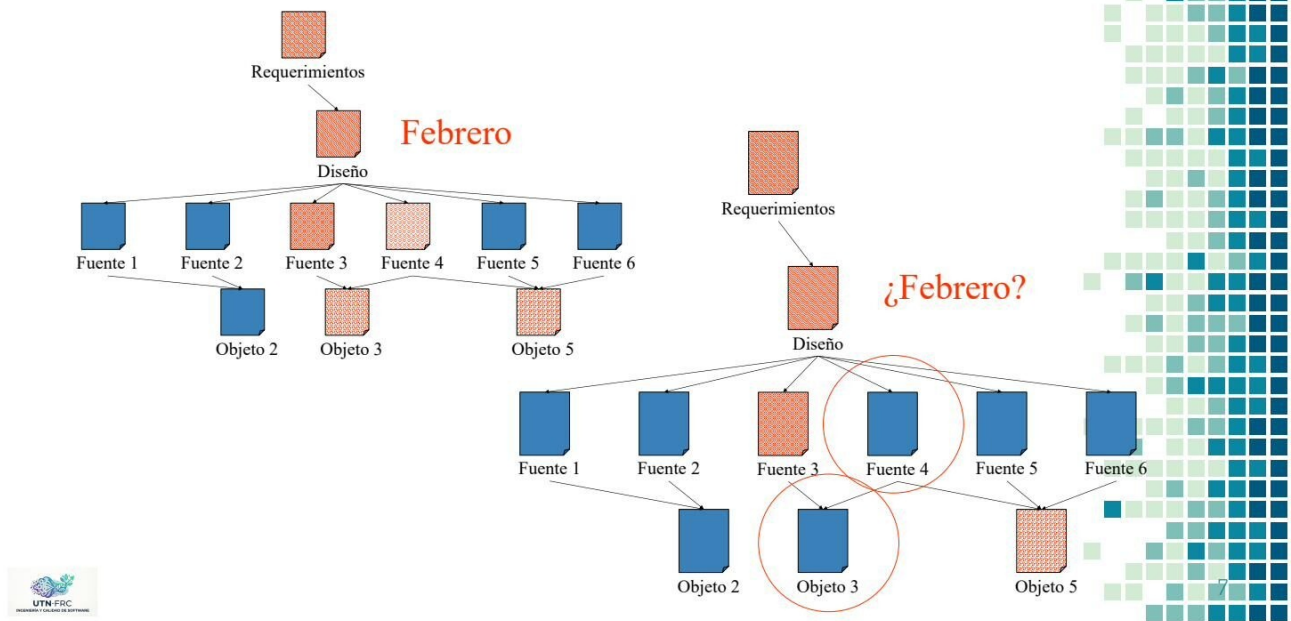


Lámina 7. ¿Febrero? Cuando algunos componentes cambiaron y otros no, la configuración deja de ser confiable.

De dónde vienen los cambios

- Cambios del negocio y nuevos requerimientos.
- Soporte de cambios de productos asociados (una nueva versión del sistema operativo, de una biblioteca, de una API externa; API es *Application Programming Interface*, **interfaz de programación de aplicaciones**: el «contrato» mediante el cual un sistema expone servicios a otro).
- Reorganización de las prioridades de la empresa por crecimiento.
- Cambios en el presupuesto.
- Defectos encontrados que hay que corregir.
- Oportunidades de mejora.

Sólo una de las seis causas es un error técnico. El resto viene del entorno, y eso conecta directamente con la propiedad de **mutabilidad** que vimos con Brooks en la Unidad 1: el software exitoso es el que más presión de cambio recibe. La lámina 14 lo resume con una imagen: **el software es un blanco móvil**. La SCM no intenta detener el cambio; intenta que el cambio no destruya la integridad del producto.

Problemas en el manejo de componentes

Son los síntomas concretos de no gestionar la configuración. Conviene saberlos con un ejemplo cada uno, porque suelen aparecer en los parciales:

PROBLEMA	QUÉ SIGNIFICA EN LA PRÁCTICA
Pérdida de un componente	Un archivo desaparece (se borró, estaba sólo en la máquina de alguien que se fue) y no hay forma de reconstruirlo.
Pérdida de cambios	El componente que tengo no es el último: trabajo sobre una versión vieja y pierdo modificaciones que otro ya había hecho.
Sincronía fuente – objeto – ejecutable	El ejecutable en producción no se corresponde con los fuentes que tengo; no puedo saber con qué código se generó.
Regresión de fallas	Un defecto ya corregido vuelve a aparecer porque alguien construyó con una versión anterior del componente.
Doble mantenimiento	Existen dos copias del mismo componente y cada corrección hay que hacerla dos veces (y tarde o temprano alguna se olvida).
Superposición de cambios	Dos personas modifican el mismo componente al mismo tiempo y el último en guardar pisa el trabajo del otro.
Cambios no validados	Se introducen cambios sin revisión ni prueba, directamente sobre lo que ya estaba aprobado.

03 La SCM como disciplina de soporte

Lám. 9–11

En la Unidad 1 la materia se organizó en disciplinas técnicas, de gestión y de soporte. La SCM es una disciplina de **soporte**, y la lámina 9 la define como una **actividad «paraguas»**: transversal a todo el proyecto y relevante para el producto a lo largo de **todo su ciclo de vida**, no sólo mientras dura un proyecto. No es una fase que se hace en un momento; cubre a todas las demás.

Disciplinas de soporte del Software

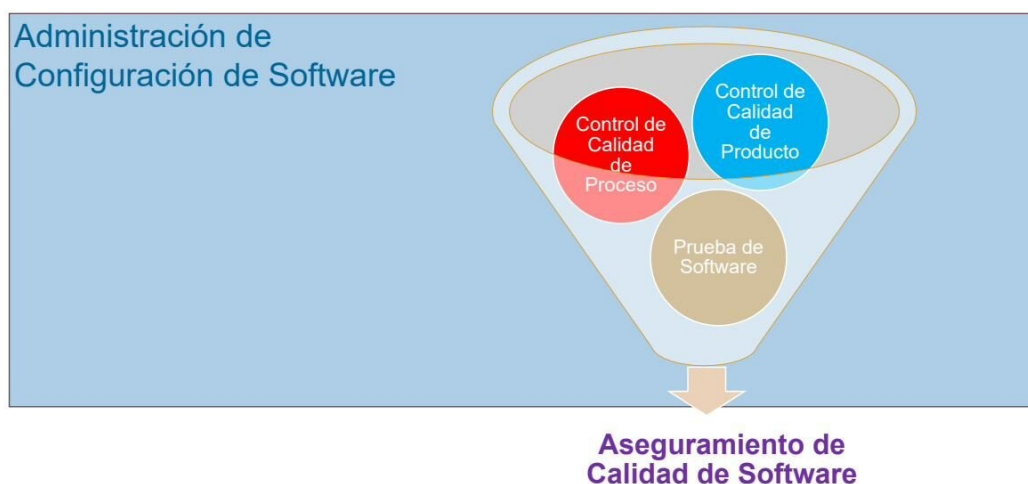


Lámina 10. Disciplinas de soporte: la gestión de configuración contiene y alimenta a las pruebas y a los controles de calidad de proceso y de producto, que confluyen en el aseguramiento de calidad.

La lámina 10 muestra un embudo: dentro de la gestión de configuración conviven la **prueba de software**, el **control de calidad de proceso** y el **control de calidad de producto**, y todo desemboca en el **aseguramiento de calidad de software**. La lectura es que **sin configuración controlada no hay calidad verificable**: no se puede probar ni auditar algo si no se sabe con certeza qué versión se está probando o auditando. Por eso la SCM es la base sobre la que se apoyan las unidades de calidad del final de la materia.

Un poco de historia

La disciplina se origina a mediados de los años 50, cuando la **CM** — *Configuration Management*, **gestión de configuración** a secas, sin el «software»— se usaba en el desarrollo de hardware y en el control de producción (típicamente en la industria aeroespacial y de defensa), y de ahí se trasladó al desarrollo de software. La foto de la lámina 11 muestra una pila de listados impresos de código más alta que la persona que la sostiene: en esa escala, perder una hoja o mezclar versiones era un riesgo muy concreto.

04 Definición y propósito: la integridad del producto

Lám. 12–14, 35

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos.

ANSI/IEEE 828, 1990

Las siglas de la fuente: **ANSI** es el *American National Standards Institute*, el **Instituto Nacional Estadounidense de Estándares** (su par argentino es el IRAM, Instituto Argentino de Normalización y Certificación); **IEEE** es el *Institute of Electrical and Electronics Engineers*, el **Instituto de Ingenieros Eléctricos y Electrónicos**, que en castellano se pronuncia «i-triple-e» y es la organización que publica el SWEBOOK (*Software Engineering Body of Knowledge*, el **cuerpo de conocimiento de la ingeniería de software** visto en la Unidad 1) y buena parte de los estándares de ingeniería de software.

La definición es larga, pero si se la lee con atención ya contiene las **cuatro actividades** de la sección 09, en orden:

FRAGMENTO DE LA DEFINICIÓN	ACTIVIDAD
«identificar y documentar las características... de los ítems de configuración»	Identificación
«controlar los cambios de esas características»	Control de cambios
«registrar y reportar los cambios y su estado de implementación»	Informes de estado
«verificar correspondencia con los requerimientos»	Auditorías

PARA ACTUALIZAR

La norma que citan las láminas es la versión de 1990. El estándar IEEE 828 fue revisado varias veces; la versión vigente es la **IEEE 828-2012**, que amplía el alcance a la gestión de configuración en ingeniería de sistemas y de software. Para el parcial la definición de 1990 es la que pide la cátedra; si la citás en un trabajo, aclará la versión.

¿Por qué gestionar la configuración?

El propósito, dicho en una línea: **establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida**. Para eso hay que poder hacer tres cosas con la configuración:

- **Identificarla** en un momento dado (saber exactamente qué la compone).
- **Controlar sistemáticamente sus cambios** (que nada cambie «porque sí»).
- **Mantener su integridad y origen** (saber de dónde viene cada cosa).

Qué significa «integridad del producto»

Un producto íntegro es el que:

- satisface las necesidades del usuario;
- puede ser fácil y completamente **rastreado** durante su ciclo de vida;
- satisface criterios de *performance* (desempeño o rendimiento);
- cumple con sus expectativas de costo.

Notá que «integridad» no es sinónimo de «que funcione». Incluye la **trazabilidad** (poder responder por qué el sistema es como es y qué cambió) y el costo. Un sistema que funciona pero del que nadie sabe con qué fuentes se construyó no es íntegro.

05 Conceptos clave: ítem, versión, variante, configuración

lám. 16–21

La lámina 16 lista los cinco conceptos que sostienen toda la unidad: **ítem de configuración, repositorio, línea base, ramas y configuración del software**. En esta sección vemos los que describen *qué* se gestiona; el repositorio, las líneas base y las ramas tienen secciones propias.

Ítem de configuración

Se llama *ítem de configuración* a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.

Lámina 17

OJO CON LAS SIGLAS: CUATRO NOMBRES PARA LO MISMO

En las láminas el mismo concepto aparece con cuatro siglas distintas. **SCI** — *Software Configuration Item* — y **CI** — *Configuration Item* — son las formas en inglés; **IC** (ítem de configuración) e **ICS** (ítem de configuración de software) son sus equivalentes en castellano. Las cuatro significan lo mismo.

Cuidado con «CI»: en ambientes ágiles y de DevOps (contracción de *Development* y *Operations*: la cultura que integra a quienes desarrollan con quienes operan el software en producción), CI casi siempre significa *Continuous Integration*, **integración continua** (ver sección 15). En un parcial conviene escribir «ítem de configuración» o «IC» para no dejar dudas.

La definición tiene tres criterios, y basta que se cumpla uno para que algo sea ítem de configuración: que **pueda cambiar**, que **deba compartirse** en el equipo, o que **necesitemos conocer su estado y evolución**. No son sólo archivos de código: la lámina 18 da una lista larga que conviene tener presente, porque en los parciales suele pedirse identificar ítems de configuración de un caso.

Plan de CM
Propuestas de cambio
Visión
Riesgos
Plan de desarrollo
Plan de calidad
Plan de integración
Planes de iteración
Registro del proyecto

Requerimientos
Arquitectura del software
Prototipo de interfaz
Guía de estilo de IHM
Estándares de codificación
Código fuente
Casos de prueba
Gráficos, íconos...

Manual de usuario
Instructivo de ensamble
Programa de instalación
Documento de despliegue
Lista de control de entrega
Formulario de aceptación

IHM es *Interfaz Hombre-Máquina*; en inglés se dice **HMI** (*Human-Machine Interface*) o, más común hoy en software, **UI** (*User Interface*, interfaz de usuario). Una guía de estilo de IHM define colores, tipografías, disposición de controles y comportamiento de las pantallas.

Ítem de Configuración de Software (SCI)

- ✓ Documentos de diseño, código fuente, código ejecutable, etc.

*Se llama **ítem de configuración (IC)** a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.*

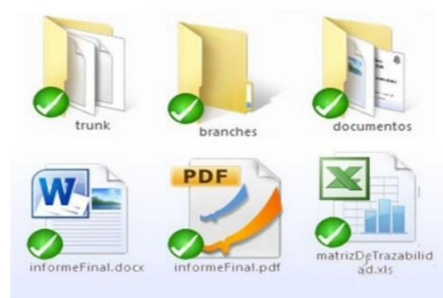


Lámina 17. Ítem de configuración: documentos de diseño, código fuente, código ejecutable, planillas... todo lo que cambia o se comparte.

Versión

Una **versión** es, desde el punto de vista de la evolución, **la forma particular de un artefacto en un instante o contexto dado**. El **control de versiones** se refiere a la evolución de **un único ítem de configuración**, o de cada ítem por separado. La evolución se representa como un grafo: en el caso más simple, una línea $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

Variante

Una **variante** es una versión de un ítem de configuración (o de la configuración completa) que **evoluciona por separado**. Representa una **configuración alternativa**: un mismo producto puede adoptar distintas formas según dónde se instale, por ejemplo según la plataforma (máquina + sistema operativo) o según qué funciones opcionales se habiliten. En el grafo, una variante es una bifurcación que sigue su propio camino ($1.3 \rightarrow 2.1 \rightarrow 2.2 \dots$).

Variante

- ❖ Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado.
- ❖ Las variantes representan configuraciones alternativas.
- ❖ Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale.
- ❖ Por ejemplo, dependiendo de la plataforma (máquina + S.O.) que la soporta, o de las funciones opcionales que haya de realizar o no.

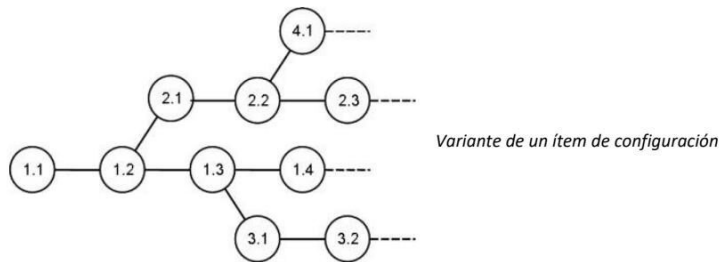


Lámina 20. Variante de un ítem de configuración: la evolución se bifurca y cada camino avanza por separado.

VERSIÓN VS. VARIANTE

Una versión **reemplaza** a la anterior en el tiempo (la 1.3 sucede a la 1.2). Una variante **convive** con otras en paralelo (la versión para Windows y la versión para Linux existen al mismo tiempo). La pregunta útil es: ¿la nueva forma sustituye a la anterior, o la acompaña?

Configuración del software

Con esas piezas se arma la definición central: la **configuración del software** es un conjunto de ítems de configuración, cada uno con su correspondiente **versión, en un momento determinado**. Es una foto: «el sistema de hoy» es la requerimientos v3 + diseño v2 + fuente A v7 + fuente B v4... Si no puedo enumerar esa lista, no sé qué configuración tengo.

06 El repositorio

Lám. 22–26

El **repositorio** es el lugar donde viven los ítems de configuración. Según la lámina 22:

- Contiene los ítems de configuración.
- Mantiene la **historia de cada ítem**, con sus atributos y sus relaciones.
- Se usa para hacer **evaluaciones de impacto** de los cambios propuestos (si sé qué depende de qué, sé qué se rompe si cambio algo).
- Puede ser una o varias bases de datos.

La lámina 23 muestra algo que parece trivial: una estructura de carpetas en un explorador de archivos. El mensaje es que **la estructura del repositorio se diseña**; no es un volcado desordenado de archivos. Esa estructura se define en la actividad de identificación (sección 10).

Funcionamiento del Repositorio

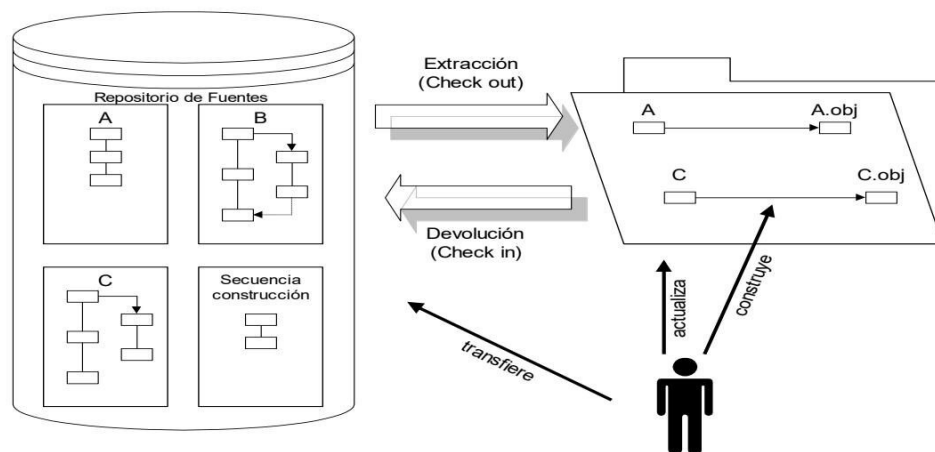


Lámina 24. Extracción (check-out), modificación, devolución (check-in) y construcción según la secuencia definida.

El ciclo básico tiene tres operaciones. La **extracción** (*check-out*) toma una copia de los ítems a modificar desde el repositorio al espacio de trabajo del desarrollador. La **devolución** (*check-in*, hoy más conocida como *commit*) sube los cambios al repositorio, que actualiza la historia del ítem. La **construcción** (*build*) genera los objetos y ejecutables a partir de los fuentes siguiendo una **secuencia de construcción** que también está guardada en el repositorio. Por eso el subtítulo de la presentación es «más allá del commit, update»: esas operaciones son la parte visible y mecánica, no la disciplina completa.

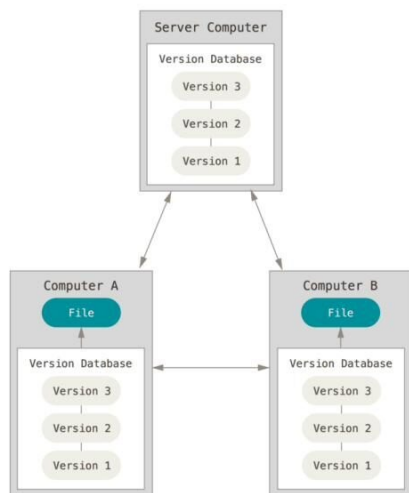
Centralizado vs. descentralizado

	CENTRALIZADO	DESCENTRALIZADO (DISTRIBUIDO)
Estructura	Un único servidor contiene todos los archivos con sus versiones; los clientes toman copias de trabajo.	Cada cliente tiene una copia exactamente igual del repositorio completo, con toda la historia.
Ventaja	Los administradores tienen mayor control sobre el repositorio.	Si un servidor falla, es cuestión de «copiar y pegar» desde cualquier copia. Habilita flujos de trabajo que no existen en el modelo centralizado.
Riesgo	Si falla el servidor, «estamos al horno»: nadie puede versionar ni recuperar historia.	Más difícil imponer control central; requiere acordar convenciones de trabajo.
Ejemplos	Subversion (SVN), CVS	Git, Mercurial

Las siglas de la tabla y de las figuras: **VCS** es *Version Control System*, **sistema de control de versiones** (la herramienta, no la disciplina); **SVN** es la abreviatura habitual de Subversion; **CVS** es *Concurrent Versions System*, «sistema de versiones concurrentes», uno de los primeros VCS populares. Cuando se habla de un VCS distribuido se usa **DVCS** (*Distributed VCS*).



Repositorios Descentralizados



- ❖ Cada cliente tiene una copia **exactamente** igual del repositorio completo.
- ❖ Si un servidor falla sólo es cuestión de "copiar y pegar".
- ❖ Posibilita otros workflows no disponibles en el modelo centralizado.

Lámina 26. Repositorio descentralizado: cada computadora tiene la base de versiones completa.

07 Líneas base

lám. 27-31

Una **línea base** (en inglés *baseline*) es:

- Una configuración que ha sido **revisada formalmente** y sobre la que se ha llegado a un **acuerdo**.
- Sirve como **base para desarrollos posteriores** y **sólo puede cambiarse a través de un procedimiento formal de control de cambios**.
- Permite **ir atrás en el tiempo** y reproducir el entorno de desarrollo en un momento dado del proyecto.

La diferencia con una configuración cualquiera es el acuerdo. Todos los días el sistema tiene una configuración; sólo algunas de esas configuraciones se «congelan» como línea base porque pasaron una revisión. A partir de ese momento, tocar algo que está en la línea base deja de ser una decisión individual del desarrollador.

Baseline (Línea Base)

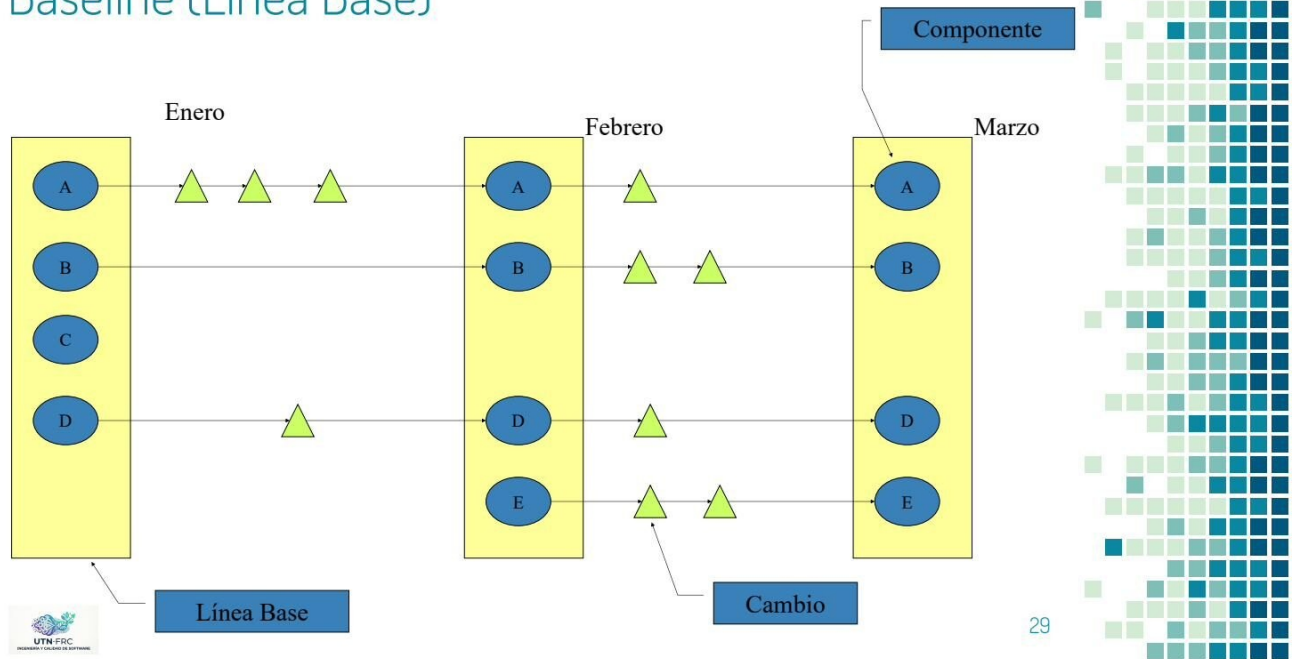


Lámina 29. Líneas base en enero, febrero y marzo: entre una y otra los componentes acumulan cambios, aparecen componentes nuevos y otros dejan de formar parte.

Cómo se marcan

Se usan **etiquetas** (*tags, labels*) para «marcar» una línea base en el repositorio: se le pone un nombre a la combinación exacta de versiones de todos los ítems en ese momento. La lámina 27 hace una advertencia que suele tomarse en los parciales:

NO CONFUNDIR

La etiqueta de una línea base **no es la versión del producto**. Un proyecto puede tener muchas líneas base internas (la de requerimientos, la de diseño, la del cierre de cada iteración) antes de liberar una sola versión al cliente. La versión del producto (por ejemplo «2.1») es lo que se comunica hacia afuera; la línea base es un punto de control interno.

Tipos de líneas base

Según la lámina 30 pueden ser:

- **De especificación:** requerimientos, diseño. Todavía no hay código, pero hay un acuerdo sobre qué se va a construir.
- **De productos** que han pasado por un control de calidad definido previamente: código probado, entregables aceptados.

El esquema clásico asocia una línea base al final de cada etapa: de planificación, de requerimientos, de diseño, de codificación, de prueba y, al final, la **línea base de configuración del software** que entra en mantenimiento.

Evolución de una configuración

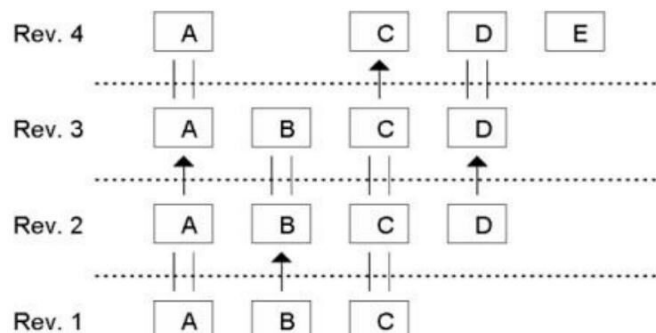


Lámina 31. Evolución de una configuración a lo largo de cuatro revisiones: se agregan, modifican y retiran ítems.

De la revisión 1 a la 4 la configuración pasa de {A, B, C} a {A, C, D, E}: se incorporan D y E, se modifican ítems existentes y B deja de formar parte. Cada revisión es una configuración distinta y el repositorio permite reconstruir cualquiera de ellas.

08 Ramas

Lám. 32–34

Creación de ramas

Una **rama** (*branch*) es una línea de desarrollo paralela dentro del repositorio.

- Existe una **rama principal**, históricamente llamada *trunk* («tronco») y en Git *master* o, en repositorios más nuevos, *main*.
- Sirven para **bifurcar el desarrollo**.
- Pueden tener **razones de creación con semántica**: una rama por funcionalidad, por corrección urgente, por versión liberada.
- Permiten la **experimentación** sin poner en riesgo la rama principal.
- Pueden ser **descartadas o integradas**.

En el ejemplo de la lámina 33 la rama principal avanza $C0 \rightarrow C1 \rightarrow C2$ y desde $C2$ sale una rama *iss53* (por «issue 53», una incidencia) con su propio commit $C3$. El nombre de la rama ya dice para qué existe: eso es la «semántica» de la creación.

Integración de ramas



- ❖ La operación se llama merge
- ❖ Lleva los cambios a la rama principal
- ❖ Pueden surgir conflictos (resolvemos con diff)
- ❖ Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas



Lámina 34. La integración de ramas: el chiste gráfico es la integral ($\int$) de las ramas, «d-rama».

- La operación se llama **merge** (en castellano, **fusión** o, más habitual en la jerga, «mergear»).
- Lleva los cambios de una rama a la rama principal.
- Pueden surgir **conflictos** cuando dos ramas modificaron la misma parte de un archivo; se resuelven con **diff** (la herramienta que muestra las **diferencias** línea por línea entre dos versiones).
- Todas las ramas deberían, eventualmente, **integrarse a la principal o ser descartadas**. Una rama que vive para siempre sin integrarse es exactamente el problema de «doble mantenimiento» de la sección 02.

RAMA VS. VARIANTE

Son conceptos emparentados pero no idénticos. La **variante** es un concepto del *producto*: dos formas alternativas que conviven (versión móvil y versión web). La **rama** es un mecanismo del *repositorio* para desarrollar en paralelo, y en general está pensada para volver a la principal. Una variante suele implementarse con una rama que no se integra, pero la mayoría de las ramas no son variantes.

09 Las cuatro actividades de la SCM

Lám. 35–36

La lámina 35 repite la definición del estándar y la lámina 36 la convierte en el esquema que hay que saber de memoria: la SCM tiene **cuatro actividades fundamentales**.

Actividades Fundamentales de la Gestión de Configuración de Software

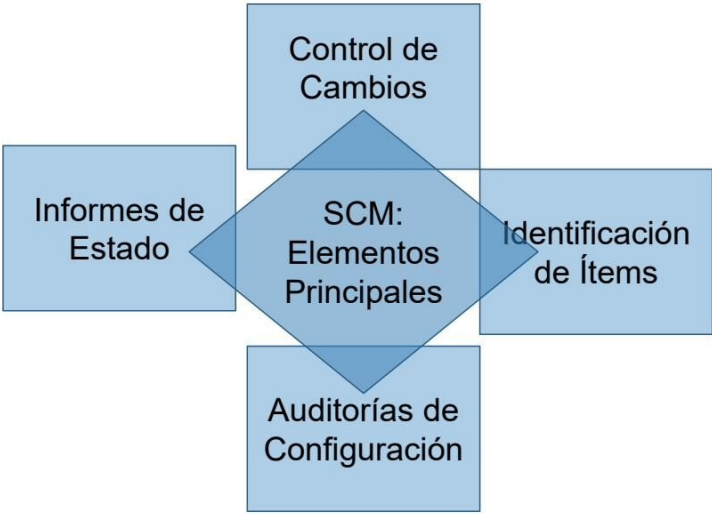


Lámina 36. Elementos principales de la SCM: identificación de ítems, control de cambios, auditorías de configuración e informes de estado.

ACTIVIDAD	PREGUNTA QUE RESPONDE
Identificación	¿Qué cosas vamos a controlar, cómo se llaman y dónde se guardan?
Control de cambios	¿Quién y cómo decide si algo que está en una línea base puede cambiar?
Auditorías	¿Lo que dice la línea base que hay, está realmente? ¿Hace lo que tiene que hacer?
Informes de estado	¿En qué estado está cada ítem y cada cambio, y cómo llegamos hasta acá?

10 Identificación de ítems de configuración

lám. 37–39

Es la actividad que se hace primero, porque no se puede controlar lo que no está identificado. Comprende:

- **Identificación unívoca** de cada ítem de configuración: cada uno tiene un identificador que no se repite.
- **Convenciones y reglas de nombrado**: cómo se forma ese identificador (proyecto, tipo de artefacto, número, versión).
- **Definición de la estructura del repositorio**: el árbol de carpetas.
- **Ubicación** de cada ítem dentro de esa estructura.

Un ejemplo de regla de nombrado: <proyecto>_<tipo>_<nombre>.<ext>, con tipos como ERS, PLN (plan), MAN (manual), y la versión administrada por la herramienta y no escrita a mano en el nombre del archivo.

Ítems de Configuración para un proyecto de desarrollo de software



Lámina 39. Ítems de configuración de producto, de proyecto y de iteración.

La lámina 39 agrupa los ítems según cuánto viven, y esa clasificación es útil para decidir la estructura del repositorio:

TIPO	EJEMPLOS	VIDA ÚTIL
De producto	ERS, arquitectura, código, manual de usuario	Toda la vida del producto; pasan de un proyecto a otro.
De proyecto	Plan de proyecto, cronograma	Mientras dura el proyecto; al cerrarlo quedan como registro.
De iteración	Plan de iteración, reporte de defectos	Sólo la iteración; después pierden vigencia.

ERS es la **Especificación de Requerimientos de Software**; en la bibliografía en inglés aparece como **SRS** (*Software Requirements Specification*). Esta clasificación se conecta con la Unidad 1: el ciclo de vida del producto contiene muchos proyectos, y los ítems de producto son los que sobreviven de uno a otro.

11 Control de cambios

Lám. 40–43

El control de cambios:

- Tiene su origen en un **requerimiento de cambio** sobre uno o varios ítems de configuración que **se encuentran en una línea base**.
- Es un **procedimiento formal** que involucra diferentes actores y una **evaluación del impacto** del cambio.

La condición «que estén en una línea base» es decisiva. Mientras un ítem no está en una línea base, el desarrollador lo modifica libremente (sólo lo versiona). Cuando ya forma parte de una línea base, cualquier modificación pasa por el procedimiento.

Proceso de Control de Cambios

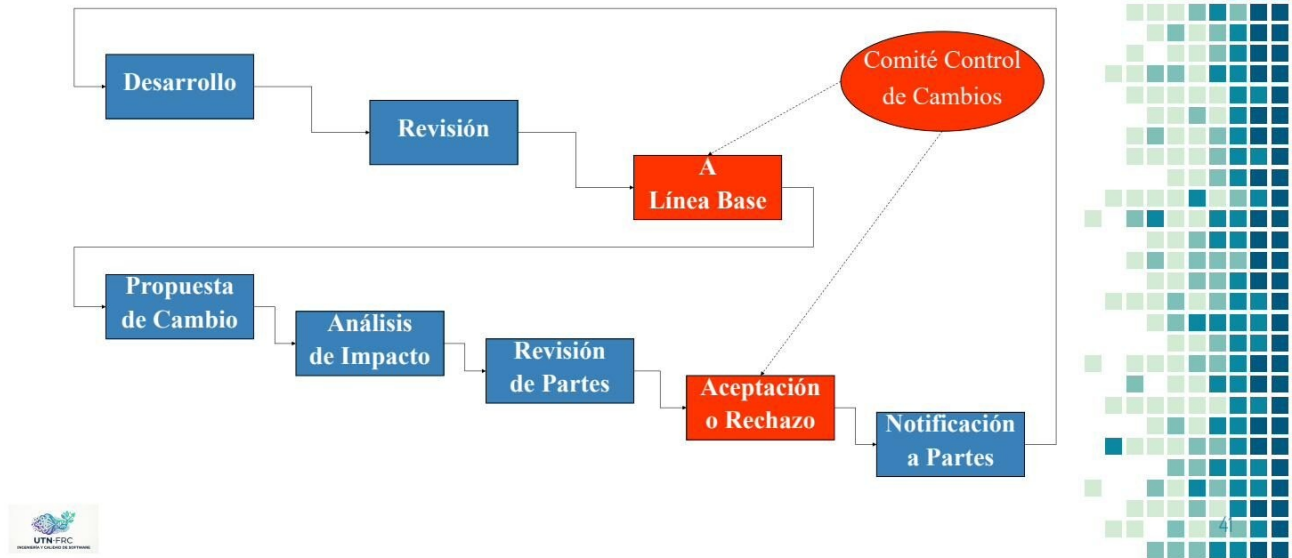


Lámina 41. Proceso de control de cambios: desarrollo, revisión y línea base; después propuesta, análisis de impacto, revisión, decisión y notificación.

El proceso paso a paso

PASO	QUÉ PASA
1 · Línea base	El ítem se desarrolla, se revisa y entra a una línea base. Desde acá, el cambio es formal.
2 · Propuesta	Alguien genera una petición de cambio (también llamada solicitud o requerimiento de cambio) describiendo qué quiere cambiar y por qué.
3 · Impacto	Se evalúa el análisis de impacto : aspectos técnicos, efectos colaterales, costos del proyecto e impacto general. Acá el repositorio y la trazabilidad son indispensables.
4 · Revisión	Revisión de las partes involucradas.
5 · Decisión	El comité de control de cambios acepta o rechaza.
6 · Notificación	Se notifica a las partes. Si se rechaza, se informa al cliente; si se acepta, se genera una orden de cambio y se ubica en la cola de cambios.

Control de Cambios

- ❖ Tiene su origen en un Requerimiento de Cambio a uno o varios ítems de configuración que se encuentran en una **línea base**.
- ❖ Es un Procedimiento formal que involucra diferentes actores y una evaluación del **impacto** del cambio

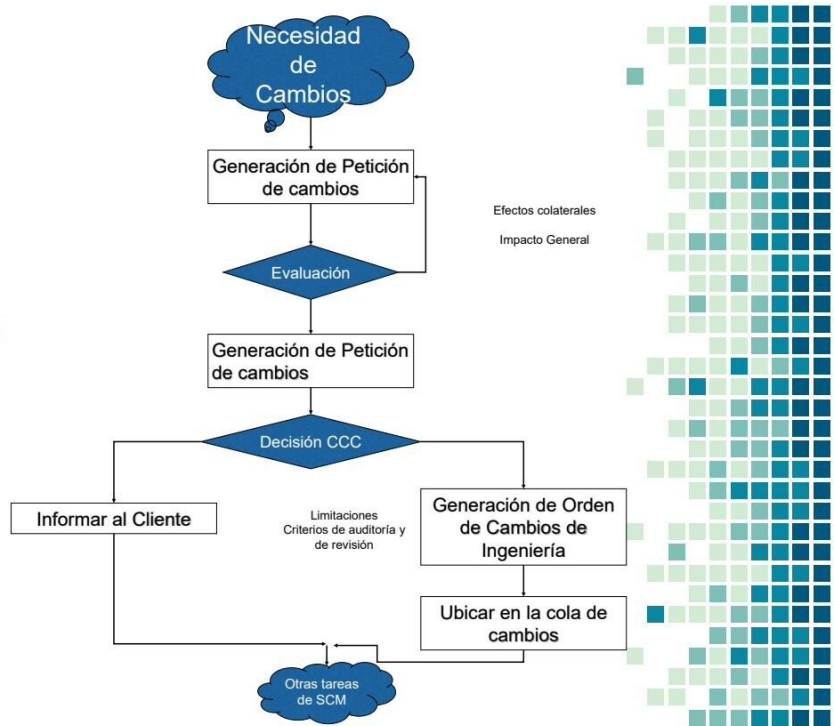


Lámina 42. Flujo detallado: necesidad de cambio, petición, evaluación, decisión del comité y orden de cambios de ingeniería.

La lámina 42 agrega un artefacto: la **orden de cambios de ingeniería**, en inglés **ECO** (*Engineering Change Order*). Es el documento que autoriza a implementar el cambio aprobado y contiene la descripción del cambio, sus limitaciones y los criterios de auditoría y revisión con los que después se va a verificar. Una vez implementado, el cambio vuelve a pasar por las demás tareas de SCM (se versiona, se incorpora a una nueva línea base, se audita y se informa).

El comité de control de cambios

Está formado por representantes de **todas las áreas involucradas** en el desarrollo: análisis y diseño, implementación, testing (prueba) y otros interesados. La idea es que nadie apruebe un cambio mirando sólo su parte: el que implementa ve el esfuerzo, el que prueba ve el riesgo de regresión, el que analiza ve el efecto sobre los requerimientos.

OJO CON LAS SIGLAS: CCC Y CCB

La lámina 42 habla de «Decisión CCC» (**Comité de Control de Cambios**) y la lámina 50 de «CCB». CCB es la sigla en inglés: *Change Control Board*. «Board» no es «tablero» ni «pizarra» sino «junta» o «comité». Las dos siglas nombran al mismo órgano.

La viñeta de la lámina 43 («¡Uf! Estuvo cerca. Casi decidimos algo.») es la advertencia sobre el riesgo del comité: si se vuelve una reunión burocrática que no decide, el procedimiento formal se convierte en un cuello de botella. Esta tensión es la que retoma la sección 15.

Auditoría de Gestión de Configuración

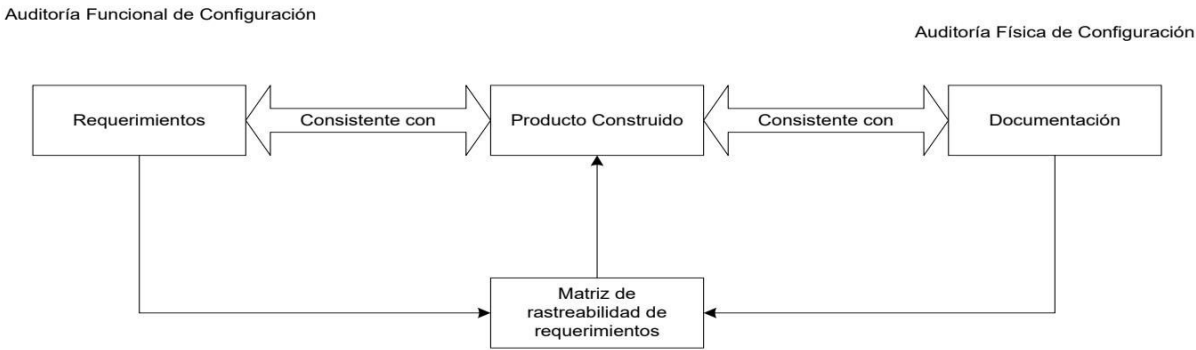


Lámina 45. La auditoría funcional compara requerimientos con producto construido; la física, producto construido con documentación. La matriz de rastreabilidad une las tres cosas.

La auditoría controla que lo que dice la línea base coincide con la realidad. Hay dos tipos, y conviene aprenderlos con la figura de la lámina 45: el producto construido está en el centro, y cada auditoría mira hacia un lado distinto.

	AUDITORÍA FÍSICA (PCA)	AUDITORÍA FUNCIONAL (FCA)
Sigla	Physical Configuration Audit	Functional Configuration Audit
Compara	Producto construido ↔ documentación	Requerimientos ↔ producto construido
Pregunta	¿Lo que la línea base (o la actualización) dice que contiene cada ítem, está realmente? ¿Están todos los ítems, en las versiones indicadas?	¿La funcionalidad y la <i>performance</i> reales de cada ítem son consistentes con la especificación de requerimientos?
Naturaleza	Inventario: presencia, versión, completitud.	Evaluación independiente del comportamiento.

Las dos auditorías se apoyan en la **matriz de rastreabilidad de requerimientos** (también llamada matriz de trazabilidad): la tabla que vincula cada requerimiento con los ítems de diseño, código y prueba que lo implementan y verifican.

Auditoría y V&V

La auditoría de configuración sirve a dos procesos básicos, que en la bibliografía se abrevian **V&V**: *Verification and Validation*, **verificación** y **validación**.

- **Validación**: el problema es resuelto de manera apropiada, de modo que el usuario obtenga **el producto correcto**.
- **Verificación**: asegura que el producto cumple con los objetivos preestablecidos, definidos en la documentación de la línea base. Todas las funciones se llevan a cabo con éxito y los casos de prueba (*test cases*) tienen estado «ok» o constan como «problemas reportados» en la **nota de release** (nota de versión o de entrega: el documento que acompaña cada liberación y lista qué incluye y qué problemas conocidos tiene).

IDEA CENTRAL

La forma clásica de recordarlo (Boehm): **verificación** responde «¿estamos construyendo el producto *correctamente*?»; **validación** responde «¿estamos construyendo el producto *correcto*?». La auditoría física está más cerca de la verificación; la funcional, de la validación.

13 Registro e informes de estado

lám. 48–50

El registro e informe de estado (en la bibliografía en inglés, *status accounting*):

- Se ocupa de **mantener los registros de la evolución del sistema**.
- Maneja mucha información y muchas salidas, por lo que suele implementarse dentro de **procesos automáticos**.
- Incluye **reportes de rastreabilidad** de todos los cambios realizados a las líneas base durante el ciclo de vida.

La mejor forma de entender para qué sirve es con las preguntas que debería poder responder:

- ¿Cuál es el estado del ítem?
- ¿Un requerimiento de cambio fue aprobado o rechazado por el comité?
- ¿Qué versión de un ítem implementa un requerimiento de cambio aprobado? (Es decir, ¿cuál es el componente que contiene la mejora?)
- ¿Cuál es la diferencia entre una versión y otra?

14 El plan de gestión de configuración

lám. 51–52

«¡También se planifica!». La SCM no se improvisa sobre la marcha; se define al inicio en un **plan de gestión de configuración** (el «Plan de CM» que aparecía como ítem en la lámina 18: el propio plan es un ítem de configuración). Debería incluir:

- **Reglas de nombrado** de los ítems de configuración.
- **Herramientas** a utilizar para SCM.
- **Roles e integrantes del comité** de control de cambios.
- **Procedimiento formal de cambios**.
- **Plantillas de formularios** (petición de cambio, orden de cambio, informes).
- **Procesos de auditoría**.

Si se lo mira con atención, el plan cubre las cuatro actividades: las reglas de nombrado son identificación; el comité, el procedimiento y las plantillas son control de cambios; los procesos de auditoría son auditorías; y las herramientas sostienen los informes de estado.

15 SCM en ambientes ágiles

lám. 53–57

La presentación cierra volviendo al Manifiesto Ágil: individuos e interacciones sobre procesos y herramientas, software funcionando sobre documentación extensiva, colaboración con el cliente sobre negociación contractual, respuesta ante el cambio sobre seguir un plan. Todo lo visto hasta acá (comité, procedimiento formal, auditorías) suena a lo segundo de cada par. La pregunta es cómo se reconcilia.

Principios de la SCM ágil

- **Sirve a los practicantes** (el equipo de desarrollo), y no al revés.
- **Hace seguimiento y coordina** el desarrollo en lugar de controlar a los desarrolladores.
- **Responde a los cambios** en lugar de tratar de evitarlos.
- Se esfuerza por ser **transparente y «sin fricción»**, automatizando tanto como sea posible.
- **Coordinación y automatización frecuente y rápida**.
- **Eliminar el desperdicio**: no agregar nada más que valor.
- **Documentación Lean y trazabilidad**: la documentación mínima que aporta valor, sin renunciar a poder rastrear los cambios.
- **Feedback continuo y visible** sobre calidad, estabilidad e integridad.

Algunos tips

- Es **responsabilidad de todo el equipo**, no de un rol aislado de «gestor de configuración».
- **Automatizar** lo más posible.
- **Educar** al equipo.
- Las tareas de SCM quedan **embebidas** en las demás tareas requeridas para alcanzar el objetivo del *Sprint* (la iteración de duración fija de Scrum; se verá en detalle más adelante en la materia).

LA IDEA DE FONDO

La SCM ágil no elimina las cuatro actividades: las **automatiza** y las **distribuye**. La identificación queda en convenciones del repositorio; el control de cambios, en revisiones entre pares y en el backlog; las auditorías, en pruebas automáticas que corren con cada integración; los informes de estado, en el historial del repositorio y en los tableros del equipo.

Para debatir: posibles respuestas

La lámina 57 deja cuatro preguntas abiertas. No hay respuesta oficial en las láminas; estas son las líneas de razonamiento más habituales, para usar como punto de partida:

PREGUNTA	UNA RESPUESTA POSIBLE
¿Qué pasa con el comité de control de cambios?	Desaparece como órgano formal. Los cambios de alcance los decide el <i>Product Owner</i> (dueño del producto) priorizando el <i>backlog</i> (la lista priorizada de trabajo pendiente); los cambios de código los aprueba el propio equipo mediante revisión entre pares (<i>pull requests</i> , solicitudes de integración).
¿Qué ítems de configuración podemos tener?	Menos documentos y más artefactos vivos: product backlog, sprint backlog, historias de usuario, definición de hecho, código, pruebas automatizadas, scripts de construcción y despliegue, configuración de ambientes.
¿Qué pasa con las auditorías?	Se automatizan en buena medida: cada integración ejecuta la construcción y las pruebas, lo que funciona como una auditoría continua. La revisión del producto con el cliente al final de cada iteración cumple parte del rol de validación.
¿Qué pasa con los reportes de estado?	Se reemplazan por información visible y automática: historial del repositorio, tableros de tareas, gráficos de avance y el estado del pipeline de integración.

SIGLAS QUE VAS A CRUZAR EN ESTE TEMA

CI/CD — *Continuous Integration / Continuous Delivery* (o *Deployment*): **integración continua** y **entrega (o despliegue) continua**. La integración continua consiste en integrar el trabajo de todos a la rama principal varias veces por día, con construcción y pruebas automáticas en cada integración; la entrega continua mantiene el producto siempre listo para liberarse. Son la forma concreta en que la SCM ágil «automatiza tanto como sea posible». Recordá que acá CI **no** es «ítem de configuración».

Una de las referencias de la presentación trata las estrategias de despliegue **blue-green** y **canary**. En *blue-green* se mantienen dos ambientes de producción idénticos («azul» y «verde»): se despliega la nueva versión en el que está inactivo y se conmuta el tráfico de golpe, con la posibilidad de volver atrás al instante. En *canary* («canario», por los canarios que se usaban en las minas para detectar gas) la nueva versión se libera primero a un porcentaje chico de usuarios y se amplía gradualmente si no aparecen problemas. Ambas dependen de saber con exactitud qué configuración está en cada ambiente, que es justamente lo que garantiza la SCM.

16 Preguntas de repaso

autoevaluación

1. ¿Cuál es el propósito de la SCM? ¿Qué significa que un producto sea «íntegro»?
2. Enumerá los problemas en el manejo de componentes y explicá dos con un ejemplo propio.
3. ¿Por qué se dice que la SCM es una actividad «paraguas»? ¿Qué relación tiene con el aseguramiento de calidad?
4. Definí ítem de configuración. Dado un proyecto que conozcas, identificá al menos tres ítems de producto, dos de proyecto y dos de iteración.
5. Diferenciá versión, variante y configuración del software.
6. ¿Qué es una línea base? ¿En qué se diferencia de una configuración cualquiera y de la versión del producto?
7. Compará repositorios centralizados y descentralizados: ventajas y riesgos de cada uno.
8. ¿Para qué sirven las ramas? ¿Qué es un merge y cuándo aparece un conflicto?
9. Nombrá las cuatro actividades de la SCM y ubicá cada una en la definición del estándar IEEE 828.
10. Describí el proceso de control de cambios. ¿Por qué sólo se aplica a ítems que están en una línea base? ¿Quiénes integran el comité?
11. Diferenciá auditoría física y funcional de configuración. ¿Qué papel cumple la matriz de rastreabilidad?
12. ¿Qué preguntas debería poder responder el registro e informe de estado?
13. ¿Qué debe incluir un plan de gestión de configuración?
14. ¿Cómo cambian el comité, las auditorías y los informes de estado en un ambiente ágil?

Referencias de la presentación

- Bersoff, E. H. «Elements of Software Configuration Management». *IEEE Transactions on Software Engineering*, vol. SE-10, n.º 1, enero 1984, pp. 79–87.
- *Little Book of Configuration Management*. SPMN — *Software Program Managers Network* (Red de Gerentes de Programas de Software, iniciativa de la Marina de los EE. UU.). spmn.com
- *SCM & the Agile Manifesto*. scmpatterns.com/agilescm/
- Harness.io. *Blue-green and canary deployment strategies*. harness.io/blog

Para ampliar

- IEEE Std 828-1990, *Standard for Software Configuration Management Plans* (y su revisión IEEE 828-2012).
- Pressman, Roger S. *Ingeniería del software: un enfoque práctico*, 7.ª edición. McGraw-Hill, 2010. Capítulo sobre administración de la configuración del software.
- Sommerville, Ian. *Ingeniería de Software*, 9.ª edición. Pearson, 2011. Capítulo sobre gestión de la configuración.
- SWEBOK v3.0 (IEEE Computer Society, 2014), área de conocimiento 6: *Software Configuration Management*.

Apunte de estudio elaborado a partir de la presentación de cátedra. Las figuras están tomadas de las láminas originales.